SCS1308 · FOUNDATIONS OF ALGORITHMS

Recursive Algorithms

Lecture 3 recap: designing, analyzing and solving recursive algorithms

Practical session summary

INTRODUCTION

Goals of the lecture

1. **Design and analyze recursive algorithms**
2. **Know when to use recursion, and when not to**
3. **Derive and solve recurrence equations**

RECURSION

What is a recursive algorithm?

Definition

A recursive algorithm solves a problem by calling itself on a smaller instance of the same problem.

- Each call works on a smaller input
- The calls stop at a base case
- The answers are combined on the way back

Two essential parts

1 Base case

Solved directly, with no recursive call. It stops the recursion.

2 Recursive case

Calls the same algorithm on a smaller size and uses its result.

```
Example: sum(n) = n + sum(n-1), sum(0) = 0
```

Example in C: sum of 1 to n

How the program works

- **sum(n)** calls itself with n − 1
- **Base case:** n ≤ 0 returns 0 with no further call
- **Recursive case:** returns n + sum(n − 1)
- **printf** lines show every call and return
- **main** starts it all with sum(4)

sum.c

```
#include <stdio.h>
int sum(int n) {
  if (n <= 0) {
    printf("sum(%d): base case, return 0\n", n);
    return 0;
  }
  printf("sum(%d): call sum(%d)\n", n, n - 1);
  int r = n + sum(n - 1);
  printf("sum(%d): return %d\n", n, r);
  return r;
}
int main(void) {
  printf("Result = %d\n", sum(4));
}
```

How it ran and what it printed

How it ran

- **Going down:** sum(4) calls sum(3), then sum(2), sum(1) and sum(0). Each call waits on the stack.
- **Base case:** sum(0) returns 0 without calling again.
- **Coming back:** the waiting calls finish in reverse order, returning 1, 3, 6 and then 10.
- **Result:** 4 + 3 + 2 + 1 = 10

Program output

```
sum(4): call sum(3)
sum(3): call sum(2)
sum(2): call sum(1)
sum(1): call sum(0)
sum(0): base case, return 0
sum(1): return 1
sum(2): return 3
sum(3): return 6
sum(4): return 10
Result = 10
```

The calls print on the way down, the returns on the way up.

SOLVING RECURRENCES

Four methods to solve a recurrence

1

Recursion tree

Draw the calls as a tree and sum the cost of each level

2

Substitution

Guess the solution, then prove it by induction

3

Iteration

Expand the recurrence again and again, then sum the series

4

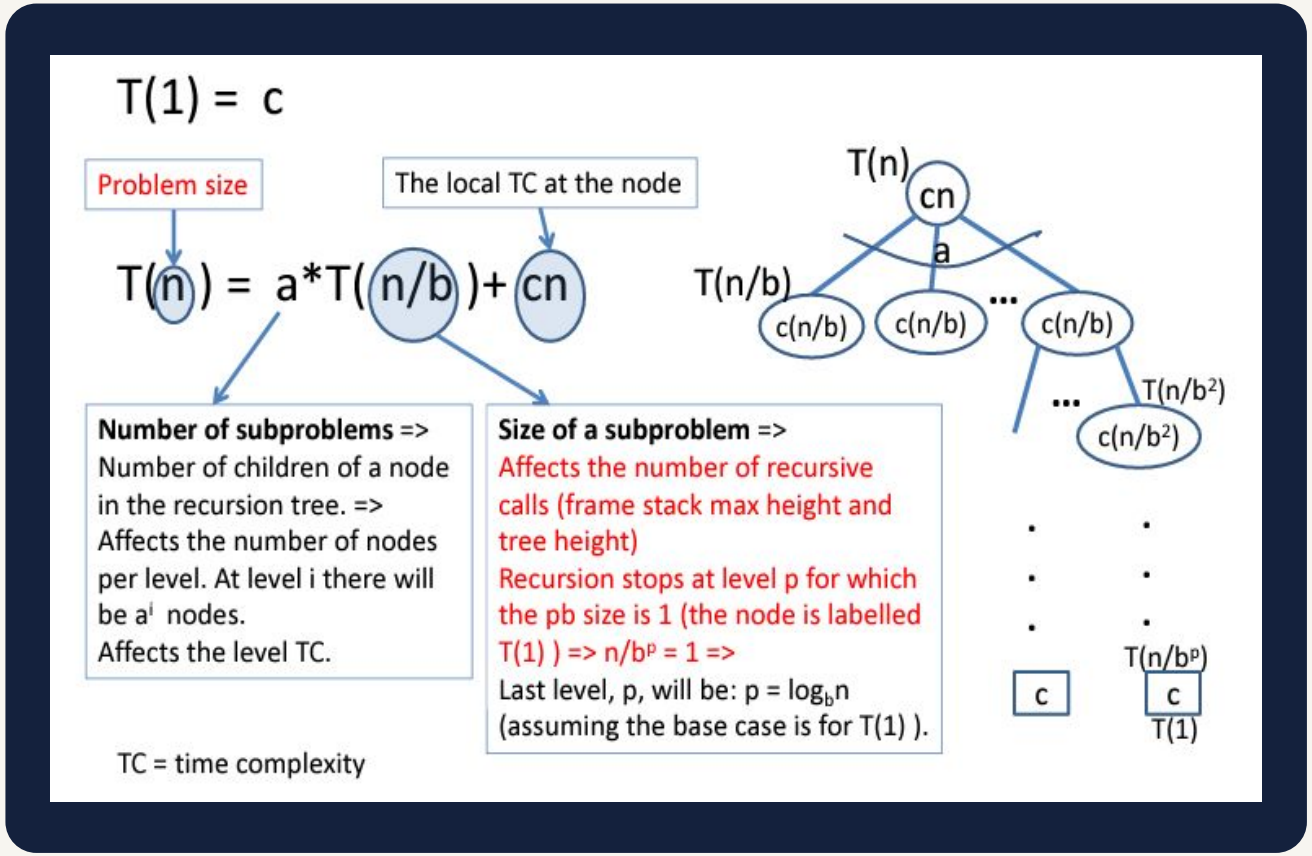
Master Theorem

Read the answer from a, b and f(n) for T(n) = aT(n/b) + f(n)

Recursion tree method

Steps

1. Draw the root and write its non-recursive cost t(n)
2. Expand each node into its recursive calls, with their sizes and costs
3. Repeat until the sizes reach the base case
4. Sum the costs at each level
5. Add the level totals to get T(n)

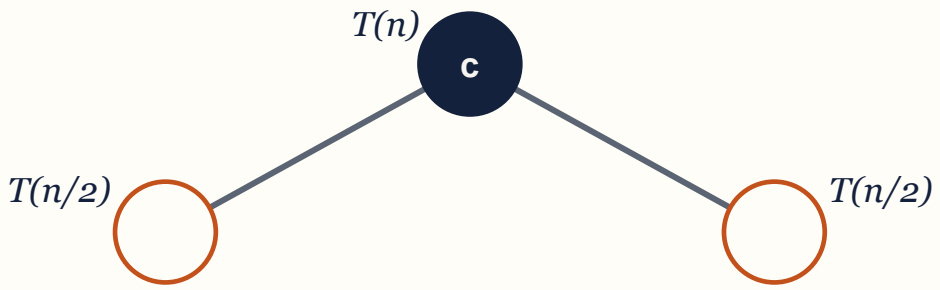


RECURSION TREE · STEP 1 OF 4

Step 1: draw the root

Base case: T(1) = c

Recurrence: T(n) = 2T(n/2) + c



Filled circle: cost known. Hollow circle: still to expand.

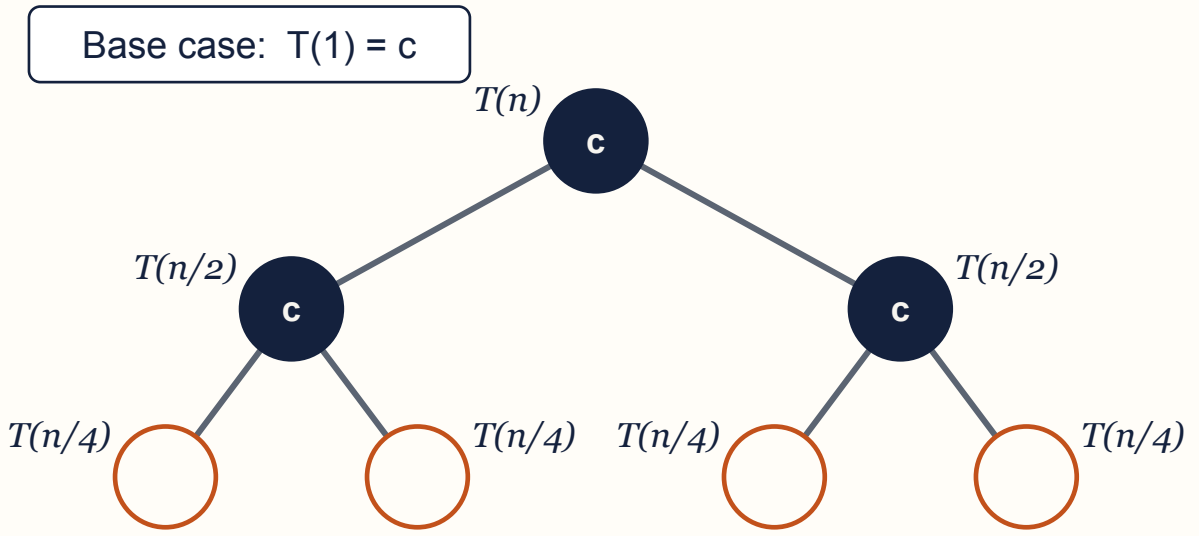
Draw the root

The cost inside the circle is the non-recursive work t(n) = c.

The recursive part, T(n/2) for each call, is written outside the circle.

The two T(n/2) circles are still empty: they are not expanded yet.

Step 2: expand each node



Recurrence: T(n) = 2T(n/2) + c

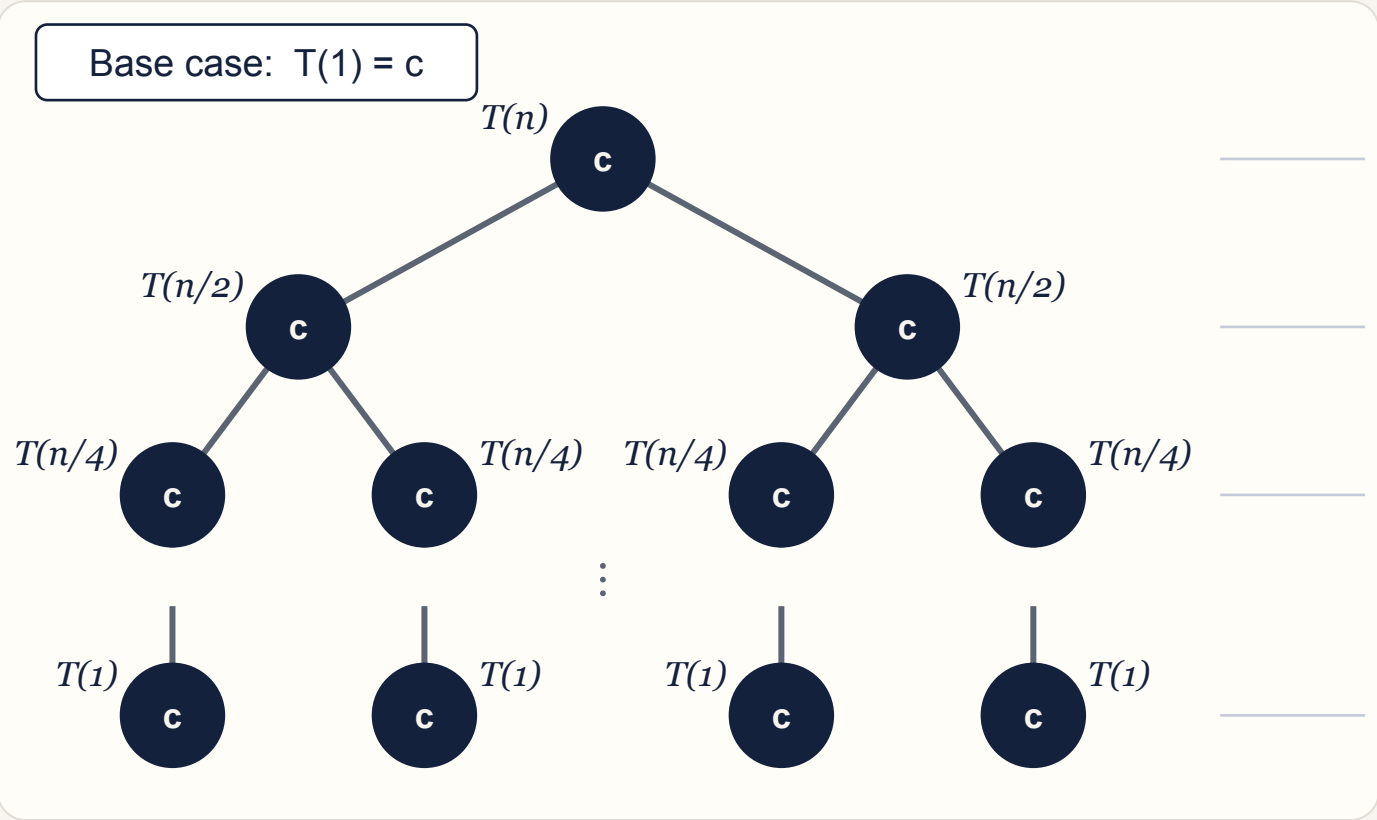
Filled circle: cost known. Hollow circle: still to expand.

Expand each node

Each T(n/2) puts c inside its circle and makes two calls T(n/4).

Level 1 costs 2 × c = 2c.

Step 3: repeat to the base case



Repeat until size 1

Level p has 2^p nodes of size n/2^p.

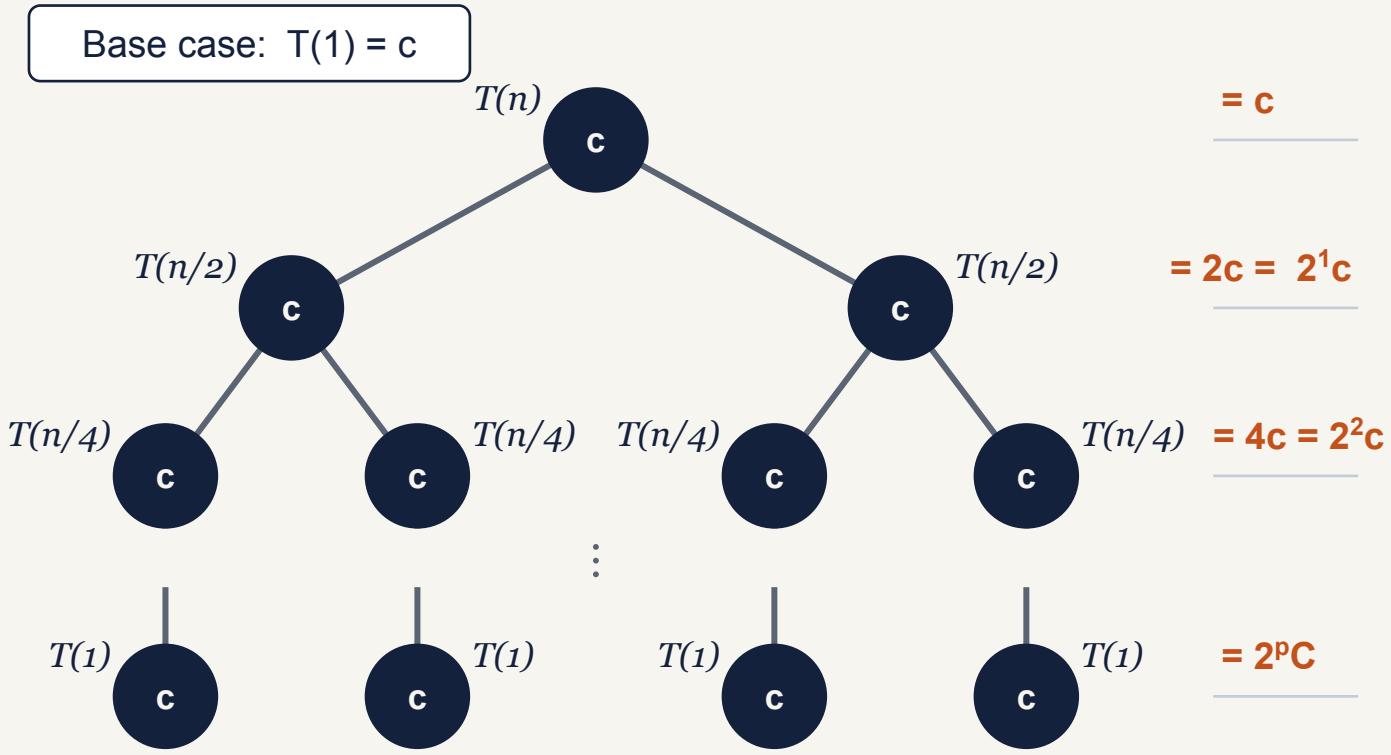
The size reaches 1 when p = lg n, where the base case T(1) = c applies.

n/2^p = 1 where p = lg n
2^p = n

So the tree has lg n levels and n leaves, each costing c.

RECURSION TREE · STEPS 4 AND 5

Steps 4 and 5: sum the levels



Add everything up

Every level costs n:

```
Level 0:    c            = 2⁰c
Level 1:    2 × c = 2c   = 2¹c
Level 2:    4 × c = 4c   = 2²c
Level p:    2ᵖ × c       = 2ᵖc
```

Total Tree Cost

$= 2^0c + 2^1c + 2^2c + 2^pc$

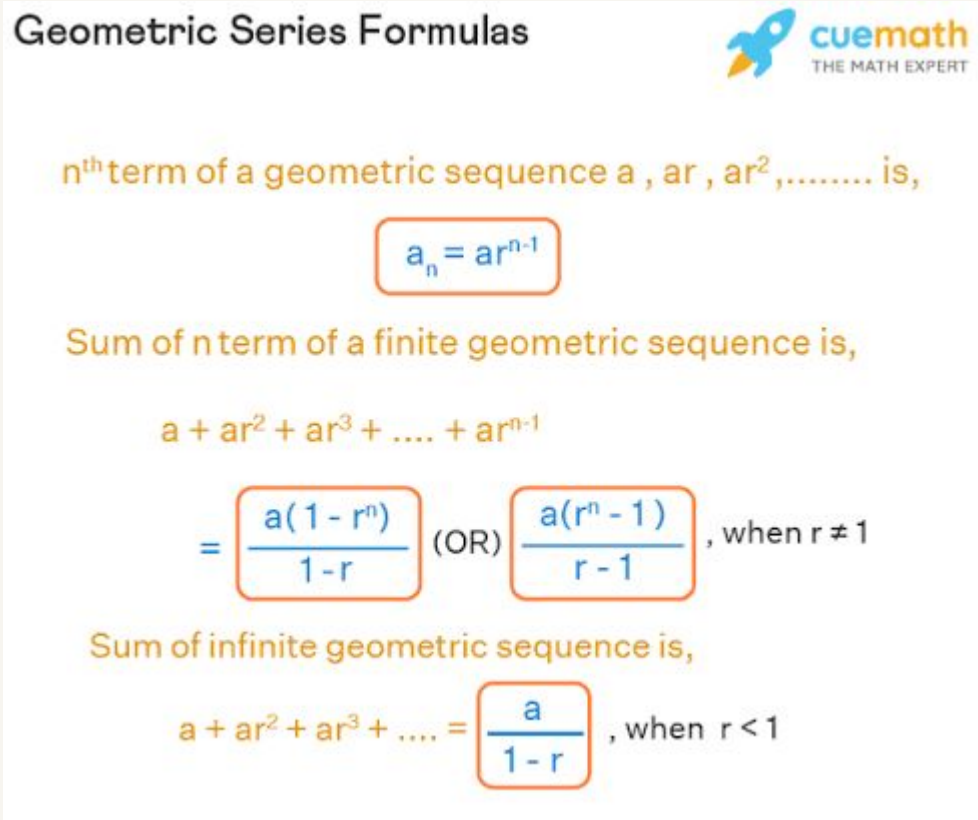
$= c(2^0 + 2^1 + 2^2 + \ldots 2^p)$

$= c2^{p+1} / (2-1)$

$= 2c2^p$

$= 2cn$

$T(n) = \Theta(n)$



Where,

a is the first term

r is the common ratio of every two successive terms

n is the number of terms.

SUMMARY

Key takeaways

1
A recursive algorithm needs recursive calls and a directly solved base case

2
Its cost is a recurrence: base case, recursive calls and non-recursive work t(n)

3
Solve it with a recursion tree, substitution, iteration or the Master Theorem

4
Sum Θ(n), fast exponentiation Θ(lg n), maximum by halving Θ(n)